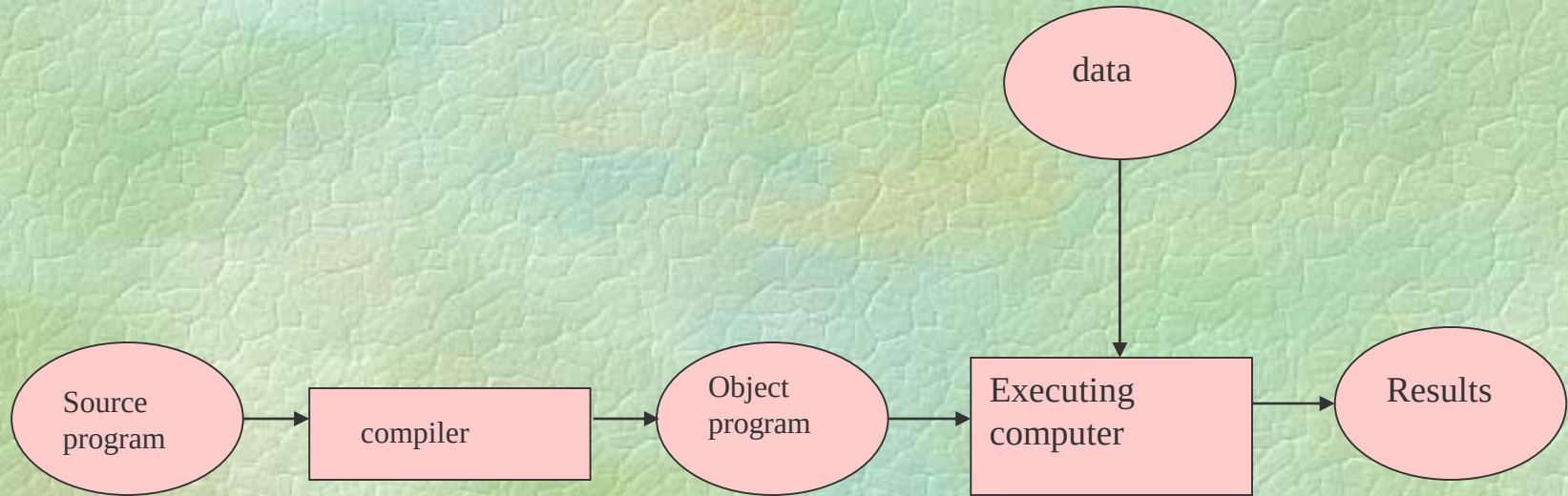


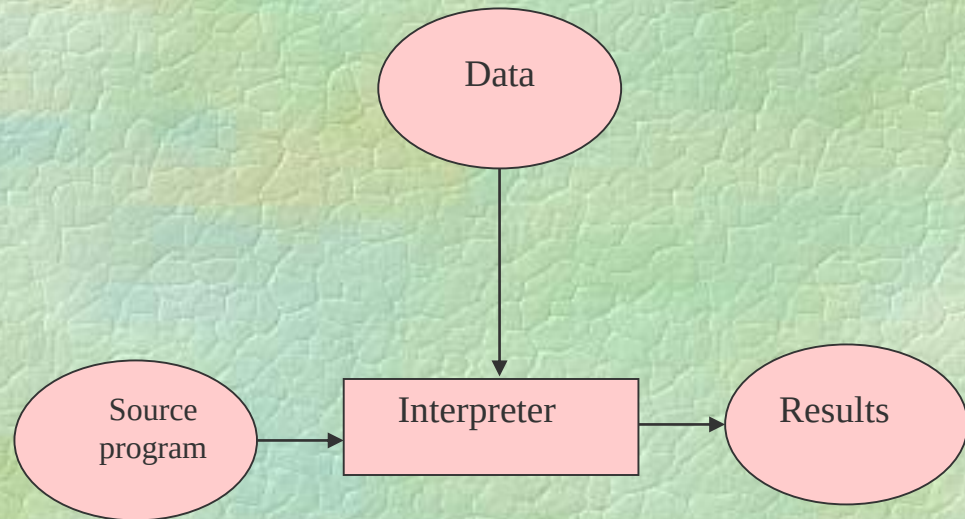
Language Processors

- In order to run /to execute/, any program written in HLL (incl. Java) should be transformed to executable form
- Three ways to convert source code into executable form:
 - Compiler - generate object code - see slide+1
 - Interpreter - interpret source code - see+2
 - Compiler of hybrid (semi-interpreting) type - +3



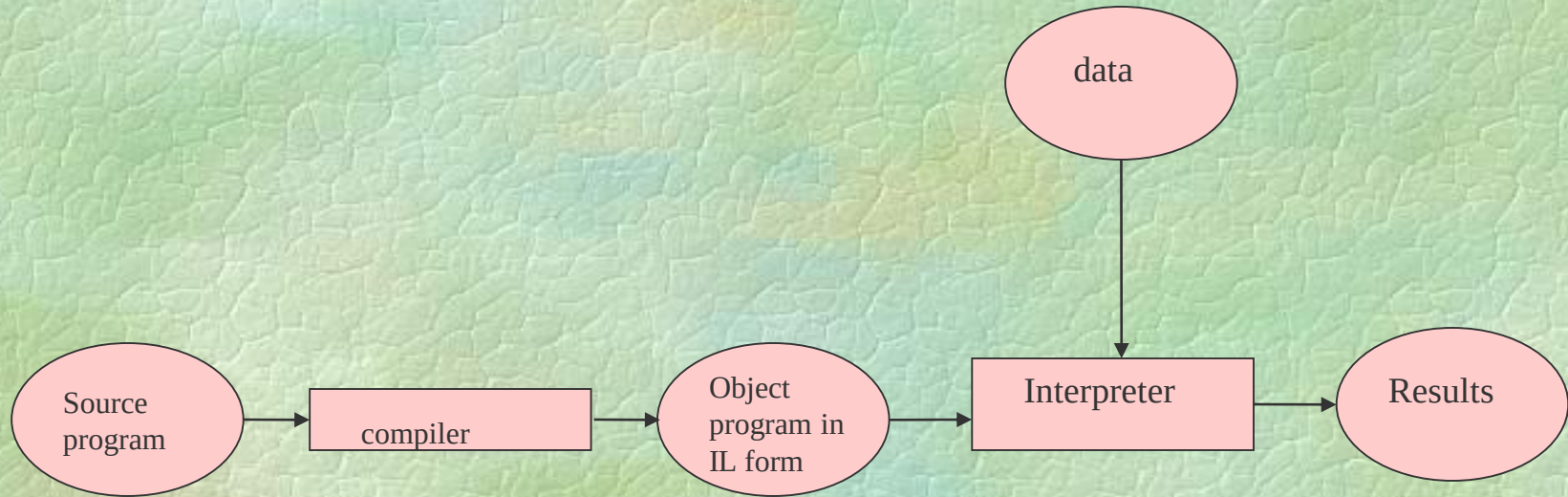
Compile time

Run time



Compile time

Run time



Compile1 time

Compile2 time

Run time

How Java works

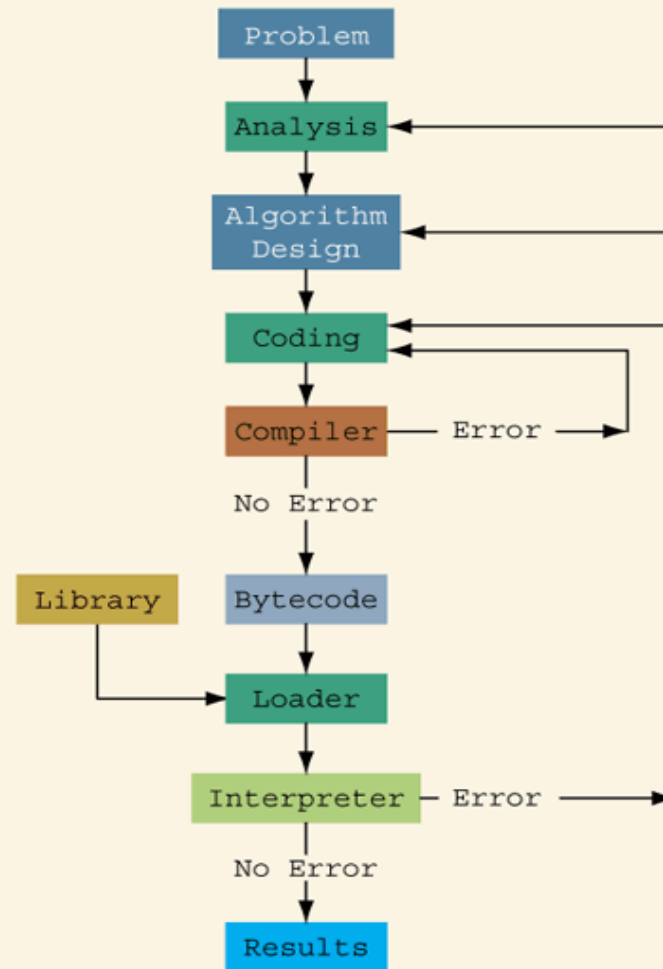
Java compiled to intermediate form - byte code.

Byte code further processed:
by interpreter named JVM
OR
by JIT compiler.

Reminder and Refresh

The SDLC concept - see next slide

Problem-Analysis-Coding-Execution Cycle



Java Components of special interest

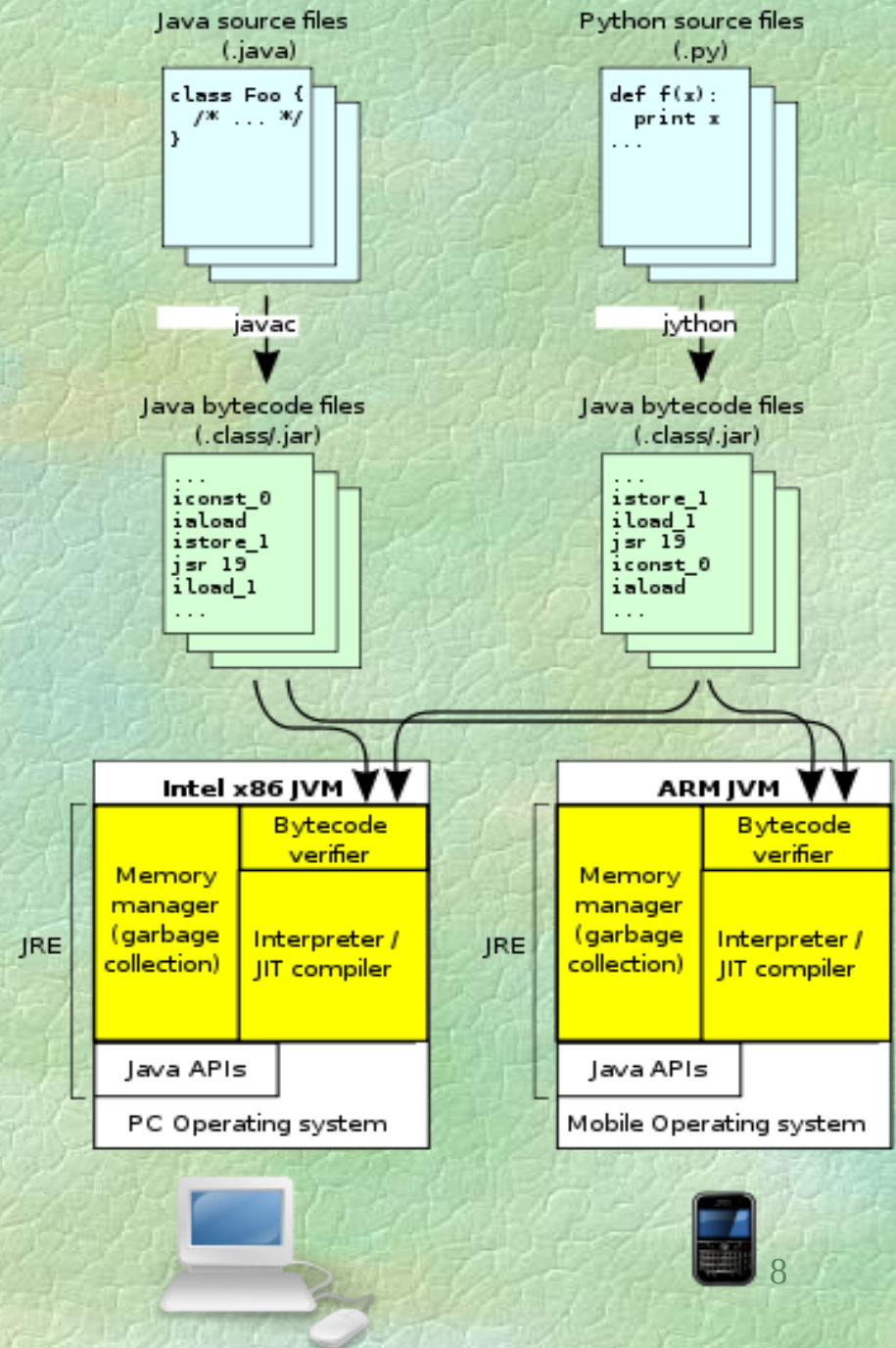
- Pure Java includes 3 software facilities:
- JRE (includes JVM)
- JVM (a part of JRE)
- JDK

JVM

JVM architecture.

Source code is compiled to Java bytecode, which is verified, interpreted or JIT-compiled for the native architecture.

The Java APIs and JVM together make up the Java Runtime Environment (JRE).



JDK contents

The JDK has
as
its primary components
a collection
of programming tools,
including:

JDK contents (most often used utilities)

- appletviewer – this tool can be used to run and debug Java applets without a web browser
- **java** – the loader for Java applications. This tool is an interpreter and can interpret the class files generated by the javac compiler.
- javac – the Java compiler, which converts source code into Java bytecode
- javadoc – the documentation generator, which automatically generates documentation from source code comments
- **jar** – the archiver, which packages related class libraries into a single JAR file. This tool also helps manage JAR files.

A Simple Java Program

Listing 1.1

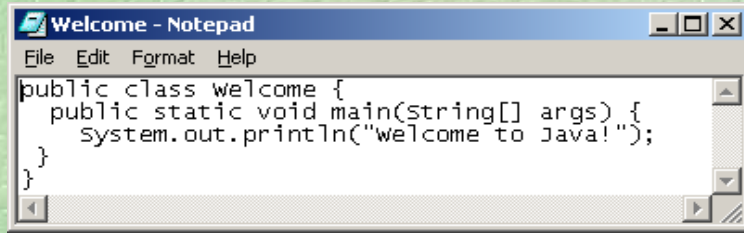
```
//This program prints message "Welcome to Java!"  
// braces in end-line style  
  
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```


A Simple Java Program

Listing 1.1

```
//This program prints message "Welcome to Java!"  
// braces in new-line style  
  
public class Welcome  
{  
    public static void main(String[] args)  
    {  
        System.out.println("Welcome to Java!");  
    }  
}
```


Creating, Compiling, and Running Java Programs



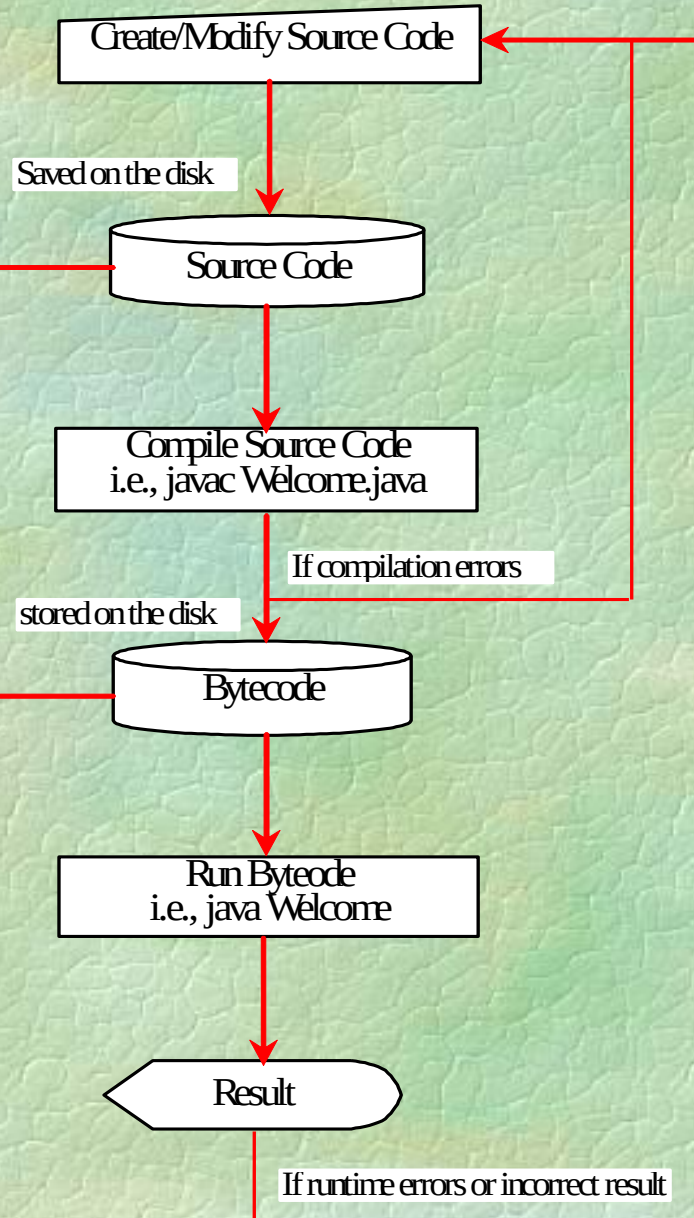
```
public class welcome {  
    public static void main(string[] args){  
        System.out.println("welcome to Java!");  
    }  
}
```

Source code (developed by the programmer)

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Byte code (generated by the compiler for JVM to read and interpret, not for you to understand)

```
...  
Method Welcome()  
  0 aload_0  
  ...  
Method void main(java.lang.String[])  
  0 getstatic #2 ...  
  3 ldc #3 <String "Welcome to Java!">  
  5 invokevirtual #4 ...  
  8 return
```



Running JAVA in Windows Environment



Creating, Compiling & Running Java Programs

- From the Command Window
 - Details follow

To open command window

- Click Start
- Select All Programs >
- Select Accessories
- Click Command Prompt

Compiling and Running Java programs

- Using any text editor , /notepad, edit, write/ type the source text of a Java demo program like this:

```
public class prog4 {  
    public static void main(String[] args) {  
        System.out.println("Hello from Java");  
    }  
}
```

- Save the Java program as a prog4.java file.

Compiling and Running Java programs

- Run compiler with statement like this:

```
javac prog4.java
```

- Run application with stmt like this:

```
java prog4
```